

Ejercicio 1:

Se interpola una función en el intervalo entre 0 y 6 utilizando cuatro nodos de Chebychev. La función es derivable con continuidad cuatro veces y la derivada cuarta toma valores entre 0 y 4. Acote el error de interpolación (en valor absoluto). Redondee a 5 decimales.



In [4]:

```
from scipy.special import factorial

M = 4 # Cota máxima para |f^{n+1}| -> me dicen que la deriv cuarta va entre 0 y 4
n = 3 # 4 nodos de Chevychev -> n + 1 = 4 -> n = 3
a = 0
b = 6

error = M * (2 / factorial(n + 1)) * ((b - a)/4)**(n + 1)

print(f'Error: {error}')
```

Error: 1.6875

Ejercicio 2:

Considere la función

$$f(x) = 1 - \cos(x)^2$$

Realice una iteración del método de Newton-Raphson partiendo del punto $x = 7.77$. De la respuesta redondeando a 5 decimales.

In [5]:

```
import numpy as np

def newton_raphson(f, deriv, x_ini, tol, maxiter, table = False):
    xk = [x_ini]
    for i in range(maxiter):
        xk = np.append(xk, xk[-1] - f(xk[-1]) / deriv(xk[-1]))
        if abs(xk[-1] - xk[-2]) <= tol: break
    return xk if table else xk[-1]

f = lambda x: 1 - np.cos(x)**2
deriv = lambda x: 2 * np.cos(x) * np.sin(x)
maxiter = 1
tol = 2
x_ini = 7.77

print(f'NR: {newton_raphson(f, deriv, x_ini, tol, maxiter).round(5)}')
```

NR: 1.83032

Ejercicio 3:

Se desea aproximar la ubicación del mínimo del polinomio

$$p(x) = x^4 - x^2 + 10$$

mediante el método de interpolación cuadrática. Determine la aproximación si el intervalo propuesto es el comprendido por los valores 0 y 0,69. Redondee la respuesta a 5 decimales.



In [6]:

```
# Acá creo que se confundió en la diapositiva y que intercambió b con c, es decir, d

f = lambda x: x**4 - x**2 + 10

a = 0
c = .69

# Como tenemos que buscar la sonrisa, y  $f(c) < f(a)$ , duplicamos h, y ahora b no esta
h = c - a
b = c + h

ya, yb, yc = f(a), f(b), f(c)
hmin = (4 * yb - 3 * ya - yc) / (4 * yb - 2 * ya - 2 * yc) * h

print(f'Mínimo: {np.round(a + hmin, 5)}')
```

Mínimo: 0.66671

Ejercicio 4:

Se utiliza el método de Heun con paso 0,04 y se obtiene el resultado 1,01. Se repite con un paso igual a la tercera parte del anterior y se obtiene el valor 1,14. Estime el error cometido con el paso original. Redondee a 5 decimales la respuesta.

In [7]:

```
x_k = 1.01
x_3k = 1.14
dt_1 = 0.04
dt_2 = dt_1 / 3

n = 2 # Heun es de orden 2

e_3k = (x_3k - x_k)/(3**n - 1) #

print(f'Error original: {np.round(e_3k * 3**n, 5)}')
```

Error original: 0.14625

Ejercicio 5:

Se sabe que una función tiene un mínimo en el intervalo entre 0 y 2. Si se utiliza el método de la razón áurea, determine el número de iteraciones necesario para obtener una aproximación con un error menor a 0,001804.



In [8]:

```
a = 0
b = 2
err = 0.001804
r = (-1 + np.sqrt(5))/2

# Usamos  $(b - a) * r^{(n + 1)} = err \rightarrow n = [\log(err / (b - a)) / \log(r)] - 1$  (chequea
print(f'Nº iter: {np.ceil(np.log(err / (b - a)) / np.log(r) - 1)}')
```

Nº iter: 14.0

Ejercicio 6:

Se utiliza el método de Heun con paso 0,05 y se obtiene el resultado 1,03. Se repite con un paso igual a la cuarta parte del anterior y se obtiene el valor 1,18. Estime el paso necesario para obtener un error menor a $1e-4$. Redondee a 5 decimales la respuesta.

In [6]:

```
maxerr = 1e-4
x_k = 1.03
x_4k = 1.18
d_t1 = .05
d_t2 = d_t1/4
n = 2

c = np.max(np.abs(x_k - x_4k) / (d_t1**n * (1 - 1/4**n)))

print(f'\u0394t: {np.round((maxerr / c)**(1/n), 5)}')
```

Δt : 0.00125

Ejercicio 7:

Se interpola la función

$$f(x) = ax^2 + 2x + 1$$

en el intervalo entre 0 y 6 utilizando un polinomio de primer orden con nodos equi-espaciados. Si a es igual a 1, encuentre el máximo error de interpolación (en valor absoluto). Redondee a 5 decimales.



In [7]:

```
f = lambda x: x**2 + 2 * x + 1
n = 1 #polinomio de primer orden con lagrange -> nodos equiespaciados
a, b = 0, 6

deriv_2 = 2 # Derivada segunda de f(x). También se puede calcular haciendo x = sp.symbols('x')
M = deriv_2 # Como deriv_2 es constante, entonces su máximo en valor absoluto en el
Q = [1, -a - b, a * b] # Es la función Q(x) = (x - a) * (x - b) = x * (x - 6)
maxabsQ = np.abs(np.polyval(Q, 3)) # Su máximo en valor absoluto estará en x = 3 (que es el punto medio del intervalo)

print(f'Cota del error: {M / factorial(n + 1) * maxabsQ}')
```

Cota del error: 9.0

Ejercicio 8:

Se utilizó el método de bisección para encontrar una solución a una ecuación no-lineal. Se sabe que, luego de 18 iteraciones, la aproximación tiene un error de, a lo sumo, 0,00072. ¿Cuál era, como mucho, la longitud del intervalo inicial? Redondee su respuesta a 5 decimales.

In [9]:

```
max_err = 0.00072
n = 18

print(f'Longitud: {max_err * 2**(n + 1)}')
```

Longitud: 377.48736

Ejercicio 9:

Suponga que se utiliza el método iterativo de Gauss-Seidel para resolver un sistema de ecuaciones. De acuerdo a los teoremas vistos en la materia, ¿para cuáles de las siguientes matrices está garantizada la convergencia?

In [9]:

```
# Verificar que sea diagonal dominante.
# O verificar que sea simétrica y definida positiva.
# O calcular  $B = (L + D)^{-1}$  y verificar que los autovalores de  $(I - B * A)$  sean meno

def conv_gauss_seidel(A):
    # Diagonal dominante
    conv1 = True
    base = np.append(False, np.full(A.shape[1] - 1, True))
    for i in range(A.shape[0]):
        idx = np.roll(base, i)
        if np.sum(np.abs(A[i, idx])) >= np.abs(A[i, i]): conv1 = False

    if conv1:
        return True

    # Simétrica y definida positiva
    if np.allclose(A, A.T) and np.all(np.linalg.eig(A)[0] > 0):
        return True

    # Método con B
    B = np.linalg.inv(np.tril(A)) #  $B = K^{-1}$ , donde K es la parte inferior de A más
    M = B.dot(A)
    eig = np.linalg.eig(np.eye(*M.shape) - M)[0]

    return np.all(np.abs(eig) < 1)

A = np.array([
    [10, -2, -2, -2],
    [-2, -10, -2, -2],
    [-1, -1, 4, -1],
    [1, 1, 1, 10]
])

print(f'Converge Gauss-Seidel: {"Sí" if conv_gauss_seidel(A) else "No"}')
```

Converge Gauss-Seidel: Sí

Ejercicio 10:

¿Cuál de las siguientes matrices no tiene descomposición Cholesky?

In [10]:

```
def has_cholesky(A):
    return np.allclose(A, A.T) and np.all(np.linalg.eig(A)[0] > 0)

A = np.array([
    [2, 2, 2, 2],
    [2, 10, 2, 2],
    [2, 2, 4, 2],
    [2, 2, 2, 1]
])

print(f'Tiene descomposición Cholesky: {"Sí" if has_cholesky(A) else "No"}')
```

Tiene descomposición Cholesky: No